TED UNIVERSITY

Department of Computer Engineering

MoonAI
Predator-Prey Neuroevolution Simulation Platform

Final Report

Team:

Caner Aras Emir Irkılata Oğuzhan Özkaya

Supervisor:

Dr. Ayşenur Birtürk

Jury Members:

Asst. Prof. Deniz Cantürk Asst. Prof. Mehmet Evren Coşkun

CMPE 492 — Senior Project II

Spring 2026

Table of Contents

1. Introduction

MoonAI is a simulation platform for studying evolutionary algorithms and neural network evolution through predator-prey dynamics. The project uses a synthetic ecosystem instead of a task-specific supervised dataset: predators, prey, and food interact in a configurable world; agent behavior is controlled by evolving neural networks; and the resulting runs produce both visual and structured data for later analysis.

The final deliverable is therefore not a single machine learning model with a fixed accuracy number. The main contribution is the simulation environment itself: a reproducible, interactive, GPU-accelerated benchmark for observing how neuroevolution behaves under changing ecological pressure. This emphasis is important because the project goal is to help study machine learning methods, not to claim that one evolved policy is the final answer to a real-world task.

1.1. Project Motivation

Modern machine learning experiments often depend on large labeled datasets or narrowly defined benchmarks. Those inputs are useful, but they also constrain what kinds of adaptation can be observed. MoonAI was motivated by a different question: can a self-generating environment be used to study how learning systems adapt, compete, diversify, and change over time?

The predator-prey setting is suitable for that purpose because it creates continuous selective pressure. Predators and prey are both required to survive, move, consume resources, and reproduce. Once those rules are combined with evolving neural networks and speciation, emergent behavior becomes an observable engineering artifact rather than an abstract concept.

1.2. Project Objectives

The project objectives were:

- implement a configurable simulation environment for predator-prey coevolution,
- integrate NEAT-style neuroevolution so both topology and weights can change over time [1],
- execute the hot simulation path on the GPU to keep large populations practical,
- provide an interactive desktop interface for experiment selection, live inspection, and settings control,
- generate reproducible output artifacts such as configuration snapshots, population metrics, species summaries, and representative genome exports,
- provide an analysis pipeline that turns run artifacts into a self-contained report.

1.3. Project Context

MoonAI was developed as a CMPE 492 senior design project. The work combines systems programming, GPU computing, simulation design, neuroevolution, UI engineering, data export, and technical documentation into a single project. The final codebase is a Rust and CUDA rewrite centered on a GPU-first runtime, with Python-based post-run analysis and a GitHub Pages documentation site.

The resulting system is useful in two ways. First, it is a research and teaching tool for studying evolutionary computation. Second, it is a software engineering case study in how to build a non-trivial simulation product around reproducibility, observability, and strong workflow discipline.

1.4. Scope of This Report

This final report covers:

- the background and design inputs that shaped the project,
- the final requirements, constraints, and scope boundaries,
- the final system architecture and simulation workflow,
- the tools and technologies used during the course,
- the executed test cases and their results,
- the engineering impact of the project in global, economic, environmental, and societal context,
- the final project status, availability links, limitations, and future work.

2. Background, Related Work, and Research Inputs

MoonAI sits at the intersection of neuroevolution, artificial life simulation, GPU computing, and experiment tooling. The final design did not come from a single paper or a single existing application. Instead, it emerged from a combination of academic ideas, internal project design iterations, and practical software engineering references.

2.1. Neuroevolution, Complexification, and Speciation

The core learning approach is NeuroEvolution of Augmenting Topologies (NEAT), which evolves both neural network weights and network structure over time [1]. This matters for MoonAI because the project is not trying to optimize a fixed neural model for a static labeled dataset. It is trying to observe how behavior changes when agents must survive in a dynamic environment and when the network itself is allowed to become more or less complex.

Two NEAT ideas are especially relevant to the final system. The first is *complexification*: networks do not need to start large. They can start from simpler forms and gain structure through mutation when the environment rewards it. The second is *speciation*: structurally different genomes can be grouped so that new innovations are not immediately eliminated by

competition with already-optimized lineages. In MoonAI, these ideas are important because predator and prey populations are both adapting at the same time. Without some form of innovation protection, short-term pressure could dominate the experiment and reduce the value of the platform as a research environment.

2.2. Predator-Prey Worlds as Research Testbeds

Predator-prey worlds are attractive research testbeds because they produce continual selective pressure. Agents do not merely solve one puzzle and stop. They must keep responding to other agents, resource availability, and reproduction constraints. The quality of a strategy is therefore relational and time-dependent. A behavior that is strong under one ecological balance may become weak after another species or another lineage adapts.

This makes the environment useful for studying:

- emergent behavior rather than hard-coded policy scripts,
- species diversity and compatibility effects,
- topology growth under ecological pressure,
- long-horizon adaptation instead of single-step optimization,
- how parameter changes alter population-level outcomes over repeated runs.

For the purposes of this project, that is more important than reporting one headline machine learning score. MoonAI is intended to be a controlled environment for observing adaptive processes, not merely a showcase for one evolved agent.

2.3. Why a Synthetic Environment Matters

The project deliberately prioritizes a synthetic environment over a pre-collected supervised dataset. That choice shapes the entire engineering direction of the final system. In a dataset-driven project, the main engineering effort often goes toward training pipelines, evaluation metrics, and model selection for one external task. In MoonAI, the engineering effort instead goes toward world rules, runtime scalability, observability, and experiment management.

This choice has several research advantages:

- the environment generates its own behavior traces and metrics,
- experiments can be rerun with fixed seeds and controlled parameter sweeps,
- selective pressure is explicit and configurable,
- the benchmark can be extended without collecting external labeled data,
- the same runtime can be used to study many conditions instead of one frozen scenario.

At the same time, the synthetic-environment approach also creates a responsibility: results must be interpreted in the context of the environment assumptions. A successful lineage in MoonAI is evidence about behavior under the rules of MoonAI. It is not automatically evidence about some different real-world task. This limitation does not weaken the project; it clarifies what kind of scientific and engineering claim the system is designed to support.

2.4. Related Work and Design Perspective

From a design perspective, MoonAI is closer to a research platform than to a narrow predictive service. It combines a simulation runtime, an evolution engine, an interactive user interface, structured data export, and a post-run analysis pipeline. Many projects use one or two of these elements in isolation. MoonAI brings them together so experiments can be configured, executed, observed, exported, and compared inside one coherent workflow.

The internal reports produced earlier in the course were also important research inputs. The project proposal established the motivation for a simulation-based learning environment. The high-level and low-level design reports helped define module boundaries and data flow. The test plan report, even though parts of the implementation changed later, still helped preserve the idea that verification must be treated as a first-class deliverable. The final report therefore builds on those earlier documents while updating them to match the final Rust and CUDA implementation.

2.5. Research Inputs Used During the Course

The final implementation was shaped by several categories of research input.

- **Academic input.** The main academic input was the NEAT literature and the broader study of adaptive multi-agent behavior in evolving environments.
- **Internal design input.** Earlier project reports and the repository documentation under `docs/` were used to preserve architectural intent and keep later code changes aligned with the project's original goals.
- **Engineering documentation.** Official documentation was used to evaluate and integrate the final technology stack.
- **Code-level exploration.** The final codebase itself served as a research artifact; it exposed what information needed to be exported, which parts of the system needed user control, and where runtime complexity was concentrated.

This mixture of sources is typical for a senior design project. The final product is not only a translation of theory into code; it is also the result of iterative design decisions made under real implementation constraints.

2.6. Use of Library and Internet Resources

Library and Internet resources were used in several concrete ways during the course.

1. to study background concepts such as neuroevolution, speciation, and evolutionary computation,
2. to evaluate technology choices for GPU execution, UI rendering, serialization, and report generation,
3. to check component APIs, version compatibility, and workflow constraints,
4. to examine examples and recommended patterns for graphics, configuration, and anal-

ysis tooling,
5. to document and visualize the final system using GitHub Pages, LaTeX, and Mermaid diagrams [12].

In practice, the team relied on official documentation and mature open-source ecosystems for:

- Rust systems programming and package management [2],
- CUDA kernel programming and host-device orchestration [3],
- immediate-mode desktop UI and GPU rendering through `egui`, `eframe`, and `wgpu` [4, 5],
- Lua-based runtime configuration through `mlua` [6],
- JSON serialization and persistence through `serde` [7],
- post-run analysis and report generation through Pandas, Matplotlib, NetworkX, and Jinja2 [8, 9, 10, 11].

The project therefore depends on both academic references and engineering documentation. The academic literature explains why the problem is worth studying; the technical references explain how to build the environment in a way that is robust, inspectable, and repeatable.

3. Requirements, Constraints, and Final Scope

The final scope of MoonAI is defined by the requirements of a simulation platform, not by the requirements of a standalone production machine learning service. The central product is an environment for running, observing, and analyzing neuroevolution experiments.

3.1. Functional Requirements

The implemented system was designed to satisfy the following functional requirements.

- **FR-1:** The runtime shall load experiment definitions from `experiments.lua` at startup.
- **FR-2:** The runtime shall persist UI-related settings through `settings.json`.
- **FR-3:** The application shall allow users to select an experiment, edit a draft configuration, queue runs, and replace the active run from the UI.
- **FR-4:** The simulation shall evolve predator and prey agents whose controllers are neural networks with mutable topology and weights.
- **FR-5:** The runtime shall render the active population and provide selected-agent inspection data.
- **FR-6:** The runtime shall export structured artifacts including `config.json`, `stats.csv`, `species.csv`, and `genomes.json`.
- **FR-7:** The project shall include an analysis pipeline that reads run artifacts and generates a self-contained HTML report.
- **FR-8:** The runtime shall support fixed seeds so experiment results can be compared

under controlled conditions.

3.2. Non-Functional Requirements

The most important non-functional requirements were:

- **Performance:** the hot simulation path should stay on the GPU so large populations remain practical.
- **Reproducibility:** experiment definitions, seeds, and output artifacts should make runs traceable and comparable.
- **Usability:** the project should provide a usable interface for selecting experiments, observing runs, and changing settings without editing source files.
- **Observability:** the runtime should expose enough metrics, snapshots, and profiler output to support debugging and analysis.
- **Maintainability:** configuration, UI, simulation, metrics, and analysis should be separated into clear modules.

3.3. Project Constraints

Several constraints strongly influenced the final design.

- **CUDA dependency:** the current runtime is centered on NVIDIA GPU execution.
- **Single-machine execution:** runs are executed locally, not through a distributed cluster scheduler.
- **Academic time budget:** the project had to prioritize a robust core runtime over optional platform features.
- **Verification cost:** GPU-first design improves runtime performance but makes step-by-step validation more demanding than a pure CPU reference implementation.
- **Documentation burden:** the project includes code, analysis tooling, documentation, and reports, so consistency across artifacts had to be managed carefully.

3.4. Final Scope Boundaries

The final scope boundaries are as important as the implemented features.

- MoonAI is **not** primarily a project about maximizing one machine learning output metric. Its main focus is the environment that supports study of evolutionary learning.
- MoonAI is **not** a generic reinforcement learning framework, cloud platform, or distributed experiment manager.
- MoonAI is **not** a production scientific cluster tool; it is a research-oriented, interactive simulation product with supporting analysis utilities.

These boundaries kept the implementation focused on one coherent goal: a high-performance simulation environment for studying neuroevolution through predator-prey dynamics.

4. Final System Architecture and Design

The final MoonAI architecture follows a GPU-first design. The CPU loads experiments and settings, starts the UI, orchestrates kernel launches, and writes compact exports. The GPU owns the dominant simulation state and performs the heaviest work for sensing, inference, movement, combat, reproduction, and metrics reduction. Because the original overview diagrams were too dense to read comfortably at report scale, the final architecture is presented below as a set of smaller and more focused flows.

4.1. High-Level Architecture

Figure 1: High-level architecture of the MoonAI platform from runtime inputs to analysis outputs.

Figure 1 shows the broad system boundary. Two runtime inputs, `experiments.lua` and `settings.json`, feed the executable. The executable exposes the experiment workflow through the UI shell and run queue, drives the GPU simulation session, produces structured artifacts, and finally hands those artifacts to the analysis pipeline.

This is the most compact way to understand the project as a whole: MoonAI is not only a simulation loop, and it is not only a report generator. It is a full experimentation workflow from configuration to runtime to analysis.

4.2. Runtime Control and Run Lifecycle

Figure 2: UI-side control flow for loading experiments, starting runs, queueing work, and recording completed sessions.

Figure 2 focuses on the application shell rather than on GPU compute. This is important because MoonAI is deliberately designed so that the user does not have to modify source code for ordinary experimentation. The user can load presets, edit a draft configuration, start immediately, queue future runs, replace the active run, and inspect completed history from within the same interface.

This UI-driven lifecycle is part of the engineering solution. Without it, the project would be a collection of kernels and export files. With it, the project becomes a usable simulation environment for research and demonstration.

4.3. Module Responsibilities

The final repository is organized around a small number of clearly scoped modules.

`main.rs`	Bootstraps the executable, resolves the runtime directory, loads settings and experiments, and starts the UI application.
`experiment.rs`	Defines `SimulationConfig`, loads `experiments.lua`, injects default values into Lua, and validates experiment values before execution.
`settings.rs`	Loads and saves `settings.json` and defines the persistent UI configuration surface.
`src/ui/`	Implements the desktop application shell, run queue, active run session, render helpers, UI state, and custom world renderer.
`src/sim/`	Defines the host-side simulation API, GPU readback structures, and CUDA bindings for the simulation and evolution kernels.
`metrics.rs`	Writes structured runtime artifacts into the experiment output directory.
`analysis/`	Discovers completed runs, groups conditions, generates plots, renders representative genomes, and builds a self-contained HTML analysis report.

This decomposition keeps configuration, runtime execution, rendering, metrics export, and analysis separate enough to remain understandable while still functioning as one coherent system.

4.4. GPU Tick Execution

Figure 3: Core compute flow of one simulation tick on the GPU-first runtime path.

Figure 3 isolates the main compute path. At each tick, the runtime builds spatial grids, computes sensor inputs, runs neural inference for both populations, updates agent state, applies movement, resolves food and combat interactions, performs reproduction, and advances counters. Presenting this separately from the UI flow makes the computational design much easier to read.

Two cadences are intentionally separated in the design:

- **`report_interval_ticks`** controls when export artifacts such as `stats.csv`, `species.csv`, and `genomes.json` are refreshed.
- **UI speed multiplier** controls how often the interactive interface refreshes the visible world state.

This separation matters because reporting and rendering serve different purposes. Export should remain consistent and comparable across runs, while UI refresh rate should remain adjustable for live observation.

4.5. Export and Analysis Flow

Figure 4: Flow from report-interval export inside the runtime to the final HTML analysis report.

Figure 4 separates the data products of the system from the live simulation path. Once a report interval is reached, compact GPU-side summaries are refreshed, host-visible data is written to disk, and the Python analysis toolchain can later discover and group valid runs for comparison. This is one of the most important architectural properties of MoonAI: each experiment is not only visualized in real time, but also preserved in a form that can be compared statistically afterward.

4.6. Design Rationale

The main architectural decisions were driven by four principles:

1. **Keep the hot path on the GPU.** Large populations make CPU-side simulation unattractive. GPU ownership of simulation state reduces host-side duplication and keeps the main path compact [3].
2. **Expose the system through a usable interface.** The project is not only a kernel benchmark. It is also a tool for selecting experiments, watching live behavior, inspecting agents, and exporting results through a desktop application built with `egui` and `wgpu` [4, 5].
3. **Separate live observation from reporting.** Adjustable UI cadence is useful for interactivity, while stable export cadence is useful for reproducibility and comparison.
4. **Make every run analyzable after execution.** Structured metrics, species summaries, and representative genomes are part of the product, not optional extras.

This combination of GPU execution, interactive runtime control, and post-run analysis is the core of the final MoonAI design.

5. Simulation Environment, Experiment Workflow, and Outputs

MoonAI is centered on the simulation environment itself. The final runtime defines how experiments are described, how they are executed, what data is produced, and how that data is reviewed after the run.

5.1. Experiment Definition and Parameter Space

The base experiment structure is defined by `SimulationConfig`. The default configuration includes a square world, separate predator and prey populations, food entities, energy rules, mutation rates, compatibility parameters, reporting cadence, and seed control. Current defaults include a world size of 3000, 12000 predators, 48000 prey, 60000 food items, and a report interval of 1000 ticks.

The shipped `experiments.lua` file expands that base into a broad experiment matrix. It defines 55 named conditions and applies 5 fixed seeds to each, producing 275 seeded runs, plus an additional `default` entry for ad hoc use. The experiment groups cover multiple kinds of study rather than only one benchmark dimension.

Table 1: Main experiment groups defined in `experiments.lua`

Group	Main variable	Purpose
Baseline sweeps	Mutation, speciation, population size	Compare broad evolutionary behavior under different default-scale pressures.
Scale experiments	Population and world size	Observe how the runtime and population dynamics behave as density and total counts change.
Density experiments	World size at fixed population	Study sparse and dense ecological pressure at the same agent count.
Run-length experiments	`max_ticks`	Compare short and long runs under otherwise similar conditions.
Energy/resource experiments	Food respawn and energy budget	Stress-test survival and reproduction under scarce or abundant resources.
Perception and mobility	Vision, speed, interaction range	Observe how sensing and movement shape survival strategies.
Complexity experiments	Add-node, add-connection, hidden-node limits	Study how network growth constraints affect behavior and topology.

5.2. Interactive Runtime Workflow

The application always launches the UI. From the interface, a user can select a preset, edit a draft configuration, queue runs, replace the current run, inspect the world, and adjust visual settings without modifying source files.

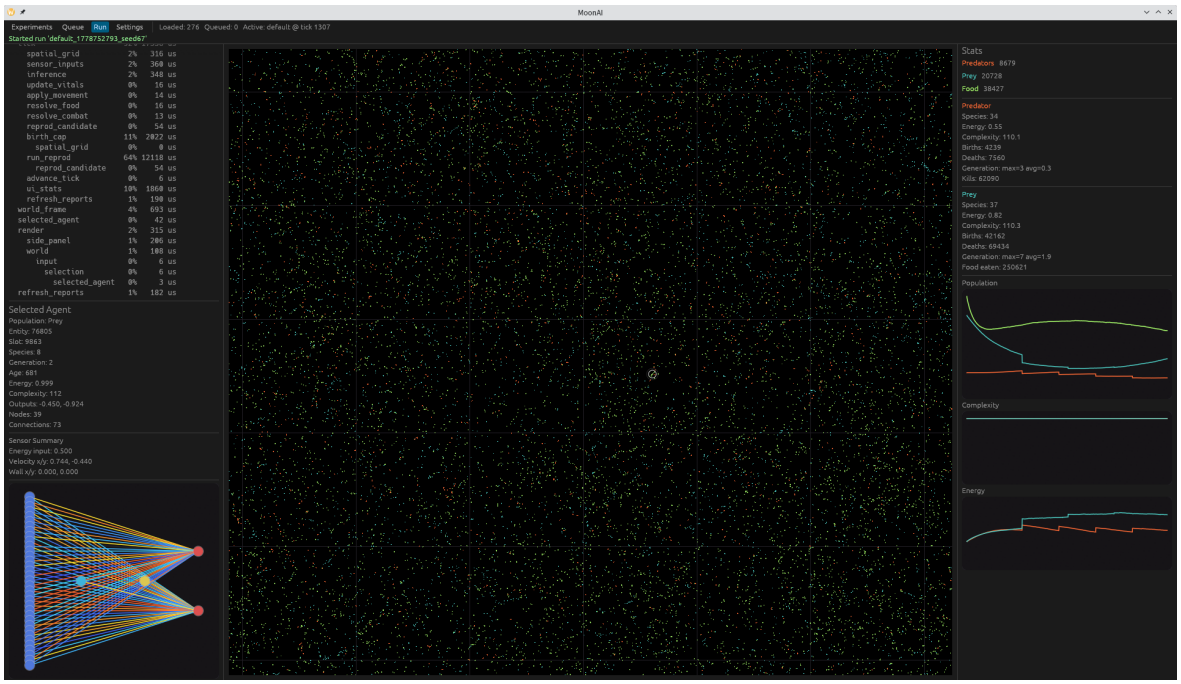

Figure 5: MoonAI desktop application showing the live simulation interface and experiment inspection workflow.

The runtime view shown in Figure 5 reflects several design decisions:

- the full active population is rendered instead of a sampled subset,
- a selected agent can expose additional information such as sensor lines and neural topology,
- queue management is built into the application shell,
- settings are persisted separately from experiment definitions.

This separation between experiment definitions and UI state is important for repeatability. Simulation parameters live in `experiments.lua`, while interface settings live in `settings.json`.

5.3. Selected-Agent Inspection and Observability

MoonAI does more than display moving points. When an agent is selected, the UI can show a vision circle, sensor lines, a stats panel, and a network panel. The controller receives 35 inputs consisting of nearest predators, nearest prey, nearest food items, self state, and wall proximity. This makes the runtime useful not only for entertainment or visualization, but for actual inspection of how evolved behavior is being produced.

The runtime also includes a scoped profiler tree. This profiler is useful when evaluating where time is spent across spatial grid construction, movement, reproduction, and other host-side control scopes.

5.4. Output Artifacts and Analysis Pipeline

Every run writes a structured output bundle under `output/experiments/`. The most important files are listed below.

Table 2: Primary experiment output artifacts

File	Purpose
`config.json`	Full configuration snapshot for the executed run.
`stats.csv`	Time-series population and complexity metrics at each report interval.
`species.csv`	Species-level population summaries over time.
`genomes.json`	Representative genome snapshots including nodes and connections.

Those artifacts feed the Python analysis package. The analysis pipeline discovers valid runs, groups them by condition, builds comparison plots, renders representative genomes, and writes a self-contained HTML report.

Figure 6: Generated HTML analysis report used to compare experiment conditions and summarize completed runs.

The report shown in Figure 6 is an important part of the final system. It turns raw exports into a usable comparison surface for researchers, which reinforces the project's main goal: providing an environment for studying evolutionary machine learning, not merely running one simulation loop.

6. Implementation, Tools, and Technologies

The final MoonAI codebase combines systems programming, GPU execution, UI rendering, serialization, experiment analysis, and documentation tooling. The value of this section is not to list technologies mechanically, but to explain why they were used.

6.1. Core Technology Stack

Table 3: Main tools and technologies used in the final implementation

Layer	Technology	Role in MoonAI
Systems language	Rust 2024	Primary implementation language for the runtime, UI, configuration, metrics, and analysis integration points [2].
GPU execution	CUDA via `build.rs` and `cc`	Executes the hot simulation, evolution, and readback path on NVIDIA GPUs [3].
Desktop UI	`eframe/egui`	Provides the application shell, controls, panels, and interactive runtime workflow [4].
GPU rendering	`wgpu`	Renders the world view and supports custom population drawing in the UI [5].
Runtime configuration	`mlua`	Loads `experiments.lua` so simulation presets remain editable without recompilation [6].
Persistence	`serde` / `serde_json`	Stores settings and exports structured artifacts in JSON form [7].
Analysis	Pandas, Matplotlib, NetworkX, Jinja2	Loads run artifacts, computes summaries, renders plots and genome diagrams, and builds self-contained HTML reports [8, 9, 10, 11].
Workflow	`just`, `uv`	Standardizes build, test, and analysis commands for the project workflow.
Documentation	Zensical, GitHub Pages, Mermaid, LaTeX	Publishes docs, renders diagrams, and produces course reports [12].

6.2. Why Rust and CUDA Were Used

Rust was selected because the final system needed strong module organization, explicit data types, good tooling, and direct interoperability with low-level code. The CUDA side was needed because the project is fundamentally population-heavy: sensor computation, inference, movement, reproduction, and compact readback become expensive when tens of thousands of agents are simulated simultaneously.

The combination allows MoonAI to keep high-level application flow readable while still moving the computationally dominant path to the GPU.

6.3. Why the UI Stack Matters

The UI stack is not cosmetic. It is part of the engineering solution. `egui` and `wgpu` were chosen because the project required:

- a desktop application with live controls,
- custom rendering for a dense simulated world,
- selection and inspection of individual agents,
- in-application queue management and settings editing.

This turns MoonAI from a raw simulation executable into an actual experimental workbench.

6.4. Why the Analysis Stack Matters

The Python analysis stack was selected to support post-run comparison rather than real-time simulation. Pandas is used for structured metric handling, Matplotlib for plots, NetworkX for topology visualization, and Jinja2 for self-contained HTML report rendering. This separation of concerns is pragmatic: Rust and CUDA handle the runtime, while Python handles data analysis and presentation efficiently.

6.5. Supporting Engineering Tools Used During the Course

Several supporting tools were also important during the course:

- **`just`** standardized recurring commands such as build, test, and analysis.
- **`uv`** simplified Python environment and dependency management for the analysis tooling.
- **Mermaid CLI** was used to generate architecture diagrams for the final report and documentation [12].
- **GitHub Pages** and Zensical were used to publish project documentation and report links.
- **LaTeX** was used to prepare the formal design and final course reports.

Together, these tools made the project easier to build, reason about, validate, and present.

7. Test Results

The final test results were recorded against the current Rust and CUDA implementation. During the report preparation process, the automated regression command `just test` was rerun on the repository snapshot and completed successfully with 7 passing automated tests and 0 failures.

Because the implementation changed substantially during the course, the final reported cases below were traced to the current codebase and final project artifacts rather than copied from an older implementation plan.

7.1. Executed Test Cases

Table 4: Final executed test cases and outcomes

ID	Test description	Result	Evidence and notes
TR-01	Metrics logger writes expected experiment artifacts.	Pass	Automated unit test in `src/metrics.rs`; confirms `stats.csv`, `species.csv`, `config.json`, and `genomes.json`.
TR-02	Profiler records nested scope trees correctly.	Pass	Automated unit test in `src/profiler.rs`; confirms hierarchical scope accounting.
TR-03	Profiler output rows preserve indentation and formatting rules.	Pass	Automated unit test in `src/profiler.rs`; confirms readable profiler formatting.
TR-04	Vision radius matches world-to-screen projection.	Pass	Automated unit test in `src/ui/render.rs`; checks selected-agent overlay math.
TR-05	Predator and prey triangle geometry remains normalized.	Pass	Automated unit test in `src/ui/world.rs`; checks renderer geometry scaling.
TR-06	Agent selection radius stays within configured bounds.	Pass	Automated unit test in `src/ui/world.rs`; checks selection interaction limits.
TR-07	Food size setting maps correctly into world-space rendering.	Pass	Automated unit test in `src/ui/world.rs`; confirms UI setting propagation.
TR-08	GUI runtime launches and renders a live experiment view.	Pass	Final screenshot evidence in Figure 5.
TR-09	Analysis report generation produces a usable HTML summary.	Pass	Final screenshot evidence in Figure 6.

7.2. Assessment of the Results

The executed final test set contains 9 recorded cases:

- 7 automated unit tests,
- 2 manual smoke checks backed by final project artifacts,

- 9 passing cases,
- 0 failing cases.

This result is strong enough to support the final submission, but it also reveals where future validation work should improve. The current automated coverage is best around metrics export, profiler formatting, and render math. Coverage is still lighter for end-to-end GPU simulation behavior, long-duration stability, and broader system smoke automation.

7.3. Bugs, Risks, and Potential Enhancements

No failing test case remained in the rerun final snapshot. However, several verification risks remain relevant:

- end-to-end kernel invariants are harder to test than pure host-side logic,
- the UI workflow still relies partly on manual validation,
- the current repository does not yet include a broad automated matrix for complete build, run, and artifact checks across multiple platforms.

The most useful testing enhancements for future work are:

1. add deterministic seed-comparison tests for compact GPU readbacks,
2. add artifact-schema regression tests for completed runs,
3. add automated smoke tests for representative experiment conditions,
4. extend manual test evidence into repeatable scripted system checks.

8. Engineering Impact, Ethics, and Contemporary Issues

MoonAI is not only a technical artifact. It also has broader engineering implications because it changes how machine learning behavior can be studied, compared, and reproduced. The impact of the project is therefore tied not only to what the simulation computes, but also to what kinds of research practice it encourages.

8.1. Global Impact

At a global level, MoonAI contributes to a broader movement toward synthetic environments and reproducible AI experimentation. A system that can generate its own evolving behavioral data is valuable for research groups that do not have access to large real-world datasets, expensive robotic hardware, or long field-collection pipelines. In that sense, the platform supports a more accessible kind of experimentation: the main challenge shifts from collecting external data to designing good experiments inside the environment.

The project also highlights an important global research trend. In many areas of machine learning, more data is treated as the main route to better results. MoonAI shows a different engineering perspective: in some settings, building a better environment is the more meaningful contribution. Better environments can expose new kinds of adaptation, failure, diversity,

and emergence that static benchmarks do not show well.

This has educational value beyond the local project team. A documented and reproducible simulation platform can serve as a shared research object. Other students or researchers can modify parameters, add new conditions, compare seeds, and extend the benchmark without starting from zero.

8.2. Economic Impact

The project was intentionally built on open-source tools and libraries. That has positive economic effects:

- lower software licensing cost,
- easier reuse by future students or research teams,
- lower onboarding cost because the stack is based on widely documented tools,
- easier long-term maintenance because the project does not depend on one proprietary runtime vendor.

MoonAI also reduces a different type of cost: experiment coordination cost. The UI, queueing model, export artifacts, and analysis report reduce the amount of manual bookkeeping needed to compare runs. That makes iterative experimentation cheaper in time, which is a real economic benefit in academic and research settings.

However, the project also makes a real economic tradeoff visible. High-performance simulation still depends on capable GPU hardware. Open-source software reduces licensing barriers, but it does not eliminate compute cost, development time, or the cost of maintaining a technically complex stack. The project is economically efficient relative to many proprietary alternatives, but it is not cost-free.

8.3. Environmental Impact

The environmental impact of MoonAI is mixed. On one hand, simulation can reduce the need for repeated physical field trials or repeated collection of external behavioral data. For some research questions, a good simulated environment can replace many low-value trial-and-error iterations that would otherwise require additional equipment use or manual setup.

On the other hand, GPU-heavy workloads consume non-trivial energy. That is especially relevant when large populations are simulated repeatedly across many seeds and conditions. In projects of this type, performance engineering has environmental consequences. Efficient kernels, compact readback, and avoiding unnecessary duplicate computation are not only speed concerns; they are also responsible engineering choices.

MoonAI partly addresses this by emphasizing compact exports and targeted analysis rather than storing every possible intermediate state. The project still uses substantial compute, but it is structured to avoid obvious waste.

8.4. Societal and Ethical Impact

MoonAI has a positive educational and research impact because it provides a controlled environment for studying learning behavior. It allows experimentation without collecting personal data by default, which is a meaningful advantage compared with many real-world AI datasets. For teaching and research, this lowers both privacy risk and data-governance burden.

At the same time, synthetic environments create a different ethical risk: overclaiming what a simulation result means. A strategy that performs well inside one benchmark should not be misrepresented as proof of general intelligence or guaranteed real-world behavior. The more convincing the visualization and metrics become, the more important this caution becomes.

There is also a fairness-related issue at the level of environment design. Any simulation bakes in assumptions about the world: what counts as success, how energy is gained or lost, how reproduction works, and what information the agents can sense. Those assumptions shape the learning outcomes. Ethical use of the platform therefore requires transparency about the rules that produce the observed results.

This is consistent with professional engineering responsibilities emphasized by ACM and IEEE ethics guidance [13, 14]:

- describe capabilities accurately,
- disclose limitations clearly,
- avoid overstating conclusions from partial evidence,
- build tools that support understanding rather than confusion,
- preserve traceability from configuration to result.

8.5. Contemporary Issues Related to the Project Topic

Several contemporary issues are directly relevant to MoonAI.

8.5.1. Reproducibility in AI Research

Many machine learning results remain difficult to repeat because configuration, preprocessing, randomness, or environment assumptions are not fully captured. MoonAI addresses this through explicit experiment definitions, seeds, output artifacts, and documentation. The platform does not solve reproducibility in general, but it does push the project in the right direction by making runs easier to trace and compare.

8.5.2. Simulation-to-Reality Gap

Synthetic environments are powerful, but their conclusions must be interpreted carefully. This is a major contemporary issue across robotics, reinforcement learning, and artificial life. MoonAI makes the gap visible rather than hiding it. The project is designed to study adaptive behavior inside the environment, not to pretend that the environment is reality.

8.5.3. Compute Concentration and Access Inequality

Advanced experimentation increasingly depends on hardware that is not equally available to all researchers. MoonAI partially lowers barriers through open-source software and self-contained documentation, but it still reflects the contemporary reality that high-performance GPU work is unevenly distributed. This is both a technical and a social issue.

8.5.4. Open-Source Governance

Research software increasingly depends on external open-source components, which makes versioning, documentation quality, and long-term maintenance important. MoonAI benefits from those ecosystems, but it also inherits their risks: upstream API changes, dependency drift, and workflow breakage must be managed responsibly.

8.5.5. Explainability of Adaptive Systems

As evolved behavior becomes more complex, observability tools such as profiler views, species summaries, network inspection, and structured exports become more important. This is a contemporary issue because many AI systems remain difficult to inspect beyond high-level output metrics. MoonAI addresses that problem by treating observability as part of the product rather than an afterthought.

8.6. Professional Responsibility

The final engineering lesson is straightforward: building the environment, documenting it, and reporting its limitations are part of the solution. For MoonAI, responsible engineering means providing a useful experimental platform while remaining honest about what the platform does and does not prove. It also means preserving enough context that future users can understand how the results were generated, what assumptions were embedded in the environment, and where the evidence stops.

9. Final Project Status, Availability, and Future Work

The final MoonAI project reached a complete prototype stage. The main runtime, UI, experiment catalog, metrics exports, and analysis pipeline are all implemented in the repository. The project is therefore finished as a senior design deliverable, while still leaving room for future research and productization work.

9.1. Final Status Overview

The final status of the project is best understood as a delivered platform with a few clearly identified extension points, rather than as a partially assembled prototype. The following work packages are complete in the final repository snapshot.

- **GPU-first simulation runtime — Complete.** Core tick execution, readback, evolution, and export refresh logic are implemented.

- **Interactive desktop application — Complete.** Experiment selection, queueing, run replacement, settings control, and selected-agent inspection are implemented.
- **Experiment configuration system — Complete.** Runtime loading from `experiments.lua`, injected defaults, and validation through `SimulationConfig` are implemented.
- **Output artifact generation — Complete.** Configuration, metrics, species, and genome exports are implemented.
- **Analysis and reporting pipeline — Complete.** HTML analysis generation from run artifacts is implemented.
- **Documentation website — Complete.** The GitHub Pages site and linked project reports are available.
- **Broad end-to-end automation — Partial.** Automated unit coverage exists, but wider system smoke automation can still be improved.

An important status point is that the final project is centered on the simulation environment. The success criterion is therefore not only whether one evolved agent looks good in one run. The success criterion is whether the platform supports repeatable experimentation, observation, artifact generation, and analysis. On that criterion, the project is complete.

9.2. Delivered Capabilities

The final delivered capabilities can be summarized as follows:

- configurable experiment definitions with large prebuilt condition sets,
- live simulation viewing with interactive controls,
- queue-based run management inside the application,
- structured export of run data for later comparison,
- post-run HTML analysis generation,
- public source and documentation availability.

These capabilities are important because they show that the final product is more than a code demo. It is a usable experimentation workflow with a documented runtime and a reproducible output format.

9.3. Project Availability

The project is publicly available through both its source repository and its documentation site.

- **GitHub repository:** `https://github.com/moon-aii/moonai`
- **GitHub Pages documentation:** `https://moon-aii.github.io/moonai/`

These links provide access to the source code, documentation, and report artifacts published

by the team. They also make the project easier to continue after the course because future users can trace both the implementation and the associated documentation from one public location.

9.4. Submission and Distribution Materials

The final project package now consists of more than the executable source code. The practical submission value comes from the combination of:

- the Rust and CUDA codebase,
- runtime assets such as `experiments.lua` and `settings.json`,
- documentation under `docs/`,
- the Python analysis package,
- the course report set under `papers/`.

This package is useful because it preserves not only the final program, but also the context required to understand, run, and extend it.

9.5. Current Limitations

The final project still has several clear limitations.

- **CUDA hardware dependence.** The runtime currently depends on CUDA-capable NVIDIA hardware for its intended hot-path execution model. This is consistent with the design goal, but it limits portability.
- **Verification depth.** The automated test suite is useful, but still lighter on full end-to-end GPU validation than on host-side logic and output formatting.
- **Single-machine workflow.** The project is optimized for local execution and study, not for distributed experiment scheduling, cluster orchestration, or shared remote services.
- **Benchmark interpretation.** The project's main focus is the simulation environment, so conclusions about the quality of individual evolved behaviors must remain conservative and environment-specific.

These limitations are important to state clearly because they define the maturity of the current system. The project is strong as a research platform and capstone deliverable, but it is not yet trying to solve every deployment scenario.

9.6. Future Work

The most valuable future extensions are listed below, with each item corresponding to an actual engineering direction rather than a generic wish list.

1. **Stronger GPU-side verification.** Add end-to-end invariant checks and determinism-oriented regression tests for compact readbacks.

2. **Artifact comparison and regression tooling.** Extend analysis support so whole experiment batches can be compared and regression-checked more automatically.
3. **Long-running suite automation.** Improve the workflow around repeated seeded runs, especially when many conditions must be executed and summarized together.
4. **Richer interactive inspection.** Expand the UI and analysis features for selected agents, species history, and population trend exploration.
5. **Extended environment variants.** Investigate additional evolutionary strategies, agent roles, or environmental rules on top of the current runtime.

Each of these directions builds on the existing platform rather than replacing it. That is a good sign for the final architecture: the current implementation is stable enough to serve as a base for later work.

9.7. Overall Final Assessment

The overall final assessment of MoonAI is positive. The project successfully delivered a high-performance simulation platform for studying neuroevolution in a predator-prey world, along with an interactive UI, structured artifacts, and analysis tooling. Just as importantly, the final result is coherent: the runtime, exports, documentation, and reports all support the same central purpose.

For a senior project, that coherence matters. MoonAI does not present a disconnected collection of technical pieces. It presents a complete experimental environment for studying machine learning through simulation. That is the main reason the project should be considered a successful final capstone outcome.

10. Glossary

10.1. Evolution Terms

NEAT NeuroEvolution of Augmenting Topologies, the evolutionary algorithm used to evolve both weights and neural network structure.

Genome The encoded neural-network representation of an agent, including node and connection genes.

Species A compatibility-based grouping of similar genomes used to preserve innovation during evolution.

Representative genome A selected genome snapshot exported to describe one species or one agent at a reporting point.

10.2. Runtime and Analysis Terms

Readback Copying compact simulation state from GPU memory to host memory for UI or export.

Report interval The tick cadence at which structured output artifacts are refreshed and written.

Speed multiplier The UI-side visualization cadence that controls how many simulation ticks are advanced per visible frame.

Render snapshot The compact host-visible population data used to draw the world in the UI.

Run queue The UI-managed list of pending experiments that execute sequentially.

GPU-first architecture A design in which the dominant simulation state and hot execution path remain on the GPU, while the CPU mainly orchestrates and exports compact results.

References

[1] K. O. Stanley and R. Miikkulainen, "Evolving Neural Networks through Augmenting Topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.

[2] The Rust Project Developers, *The Rust Programming Language*, 2026. [Online]. Available: https://www.rust-lang.org/

[3] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[4] Egui Contributors, *egui / eframe*, 2026. [Online]. Available: https://github.com/emilk/egui

[5] wgpu Contributors, *wgpu*, 2026. [Online]. Available: https://wgpu.rs/

[6] mlua Contributors, *mlua*, 2026. [Online]. Available: https://github.com/mlua-rs/mlua

[7] Serde Contributors, *Serde*, 2026. [Online]. Available: https://serde.rs/

[8] Pandas Development Team, *pandas*, 2026. [Online]. Available: https://pandas.pydata.org/

[9] Matplotlib Development Team, *Matplotlib*, 2026. [Online]. Available: https://matplotlib.org/

[10] NetworkX Developers, *NetworkX*, 2026. [Online]. Available: https://networkx.org/

[11] Pallets Projects, *Jinja*, 2026. [Online]. Available: https://jinja.palletsprojects.com/

[12] Mermaid Contributors, *Mermaid*, 2026. [Online]. Available: https://mermaid.js.org/

[13] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, 2018. [Online]. Available: https://www.acm.org/code-of-ethics

[14] IEEE, *IEEE Code of Ethics*, 2020. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html